> **Referencia en vivo y laboratorio:** para el recorrido sobre el asistente publicado y el laboratorio PostgreSQL/pgvector (`lab/`), consulta también `recorrido-cloud.md` . Este guion articula los 90 minutos de S11 sincronizados con las 16 diapositivas de `material/presentaciones/S11-crag.pptx` (diseño oscuro de thePower, con el logo en la portada). Cada diapositiva lleva además su guion breve en las notas del orador.

Guion S11 · Corrective RAG (CRAG): reordenamiento, suficiencia, grafo y métricas

Fecha: martes 27 de octubre de 2026 · **Cohorte:** M/J · **Horario:** 18:30–20:00 `Europe/Madrid` (90 minutos con preguntas).
Presentación: `material/presentaciones/S11-crag.pptx` (`material/presentaciones/S11-crag.pdf`), 16 diapositivas en cuatro secciones: Inicio (1–3), RAG avanzado y CRAG (4–12), Demo (13) y Evaluación y métricas (14–16).
Laboratorio del ejercicio: `lab/` (`bun run lab:demo` , `bun run lab:eval` , `lab/eval/resultados-laboratorio.md`).
Demostrador del curso: `bun run dev` (guiones de desarrollo locales) / `https://rag-thepower.josenuevo.dev` .

Temario de thePower para S11: RAG avanzado (superar las limitaciones del RAG básico con técnicas de autocorrección y reordenamiento de resultados) · Corrective RAG · Evaluación y métricas específicas de sistemas RAG. Ejercicio: construir un CRAG completo desde cero.

Reparto del tiempo: 20 min de conceptos (diapositivas 1–12), 55 min de demo (diapositiva 13, volviendo a la 8, la 10 y la 12 cuando haga falta) y 15 min de evaluación, ejercicio y dudas (diapositivas 14–16). El QR solo se muestra al inicio.

Nombres de los nodos: la presentación usa los del enunciado y de `lab/src/crag-graph.ts` (`retrieve` , `rerank` , `assess` , `retain` , `discard` , `reformulate` , `answer` , `refuse`). En el panel de traza del asistente del curso (`apps/api/src/crag-graph.ts`) el reordenamiento aparece como `rank` .

0–3 min · Apertura y QR de S10 (Diapositivas 1 a 3)

Diapositiva 1 · CRAG: RAG avanzado, cuando una búsqueda no basta

Explica:

> En S10 construimos un RAG de una sola búsqueda con umbral previo. Hoy evolucionamos ese mismo proyecto —conservando intacta la etapa de RAG básico como línea base— hacia un grafo de **Corrective RAG (CRAG)** en LangGraph: el sistema juzga si la evidencia cubre la pregunta y, si no, la corrige. Suficiente → responder; parcial → conservar y reformular; insuficiente → descartar y reformular. Todo con límites en el código.

Diapositiva 2 · Evalúa la clase anterior (QR)

Deja el QR visible un minuto y explica para qué sirve, como pidió thePower:

> Este QR es la encuesta de satisfacción de AI Engineer y evalúa nuestra clase anterior: **S10, la del jueves 22 de octubre**, no la de hoy. Es muy importante que la rellenéis: con vuestras respuestas ajustamos ritmo, profundidad y práctica.

Mientras responden, pide que piensen en una pregunta que el RAG de S10 contestó a medias.

Diapositiva 3 · Lo que pide el temario, en tres bloques

Presenta el temario de thePower tal como está en la ficha del programa (RAG avanzado · Corrective RAG · Evaluación y métricas específicas de sistemas RAG · ejercicio: construir un CRAG completo desde cero) y su reparto: conceptos (20 min), Corrective RAG en vivo (55 min) y evaluación y métricas (15 min).

3–20 min · RAG avanzado y CRAG: problema, técnicas, grafo, cobertura y límites (Diapositivas 4 a 12)

Diapositiva 4 · Una respuesta puede sonar bien y estar incompleta

Plantea el problema con la pregunta compuesta del guion `partial-s5-s6`: *«¿Cómo modela LangGraph el estado de un agente y cómo se observa esa ejecución?»*.

- Una única búsqueda `top-4` trae varios fragmentos de S5 y deja fuera S6 (observabilidad).
- La respuesta es fluida y cita una fuente real, pero omite la mitad: **no alucina, responde a medias**.
- En el laboratorio AetherCloud ocurre lo mismo con *«¿Cómo se autentican los clientes mTLS y qué tiempo de primera respuesta aplica en un incidente SEV-1?»* (caso 14).

Diapositiva 5 · Dónde se puede mejorar un RAG básico

Primera pieza del temario: RAG avanzado como **superar las limitaciones del RAG básico con autocorrección y reordenamiento de resultados**. Sitúa las técnicas según el momento en que actúan:

- **Antes de buscar:** reformular la consulta, descomponer preguntas compuestas, multi-query o HyDE (generar un documento hipotético y buscar con él).
- **Al buscar:** top-k amplio, filtros por metadatos (grupo, versión) y búsqueda híbrida léxica + vectorial.
- **Después de buscar:** reordenar con un reranker y evaluar si la evidencia basta; Self-RAG lleva esa crítica al propio modelo.
- **Al generar:** contexto sin duplicados, rechazar si no consta y citas verificables.

Lo resaltado en menta es lo que combina el CRAG de S11 (reformulación, top-10, reordenamiento, evaluación de la evidencia, contexto sin duplicados y rechazo). Cada técnica añade coste y hay que medirla.

Diapositiva 6 · CRAG mete un juicio entre recuperar y generar

Recorre los cuatro pasos y su nodo: recuperar (`retrieve`, hasta 10 candidatos), reordenar (`rerank`, `[0, 1]`), juzgar (`assess`, cobertura) y corregir (`retain` / `discard` → `reformulate`) o responder.

- **Qué no es:** no es GraphRAG, no entrena el modelo y no garantiza mejorar cada respuesta.
- **Alcance:** inspirado en Yan, Gu, Zhu y Ling (arXiv:2401.15884). El artículo usa un evaluador ligero, búsqueda web y descomposición del conocimiento; nosotros adoptamos la idea de evaluar y corregir sobre un corpus local. La búsqueda web queda opcional.

Diapositiva 7 · Mismo asistente, otra política de recuperación

Compara en la tabla:

- **Búsquedas:** 1 búsqueda (`top-4`) en RAG básico frente a hasta **3 búsquedas (`top-10`)** y **2 reformulaciones** en CRAG.
- **Control:** un único umbral de similitud frente a **`rerank` en `[0, 1]` + `assess` de cobertura**.
- **Contexto:** lo recuperado una vez frente a conservar, unir sin duplicados o descartar.
- **Coste:** más llamadas y más latencia.

Subraya el recuadro: corpus, índice, embeddings, modelo generativo e instrucciones son iguales en los dos; solo cambia la política de corrección.

Diapositiva 8 · El grafo de LangGraph, nodo a nodo

Abre `lab/src/crag-graph.ts` (y `apps/api/src/crag-graph.ts`) junto al diagrama:

- `retrieve` → `rerank` → `assess`; las aristas condicionales (`addConditionalEdges`) deciden la ruta.
- **Suficiente** → `answer`, que responde citando solo fragmentos aceptados en el orden validado.
- **Parcial** → `retain`; **insuficiente** → `discard`. Ambas pasan por `reformulate`, que vuelve a `retrieve` con la nueva `currentQuery`.
- **Límite:** con 3 búsquedas agotadas, o si la reformulación repite una consulta previa, el grafo termina en `refuse` **sin llamar al modelo generativo**.

Diapositiva 9 · Relevancia de un fragmento ≠ cobertura de la pregunta

Muestra los umbrales de `CRAG_SCORE_THRESHOLDS` (`packages/rag/src/crag.ts`) y `LAB_CRAG_CONFIG` (`lab/src/config.ts`), calibrados con ejemplos de validación distintos del test:

- **`minChunkRelevance = 0,50`:** un fragmento entra en la evidencia útil (`usefulIds`, máximo 6) si su puntuación de `rerank` es al menos `0,50`.
- **`partialCoverageMin = 0,40`:** por debajo, o sin ningún fragmento útil → **`insufficient`**.
- Entre `0,40` y `0,75`, o con lagunas en `gaps` → **`partial`**.
- **`sufficientCoverageMin = 0,75`** sin lagunas → **`sufficient`**.

Ejemplo de la demo: el fragmento de S5 recibe `0,80` (útil), pero la cobertura de la pregunta completa es `0,50` porque falta observabilidad → `partial`, nunca `sufficient`. Recuerda que una puntuación no es una probabilidad de acierto. Si preguntan por `Qwen3-Reranker-0.6B`, enseña `formatQwen3RerankerPrompt` en `lab/src/reranker.ts` con la plantilla oficial `yes` / `no`.

Diapositiva 10 · Parcial conserva; insuficiente descarta

Recorre las dos columnas con los guiones que se ejecutarán en la demo:

1. **Parcial (`partial-s5-s6`, «Evidencia parcial y segunda búsqueda»):** la búsqueda 1 trae `s05-langgraph:00` (`0,800`); `assess` detecta la laguna (*«Falta cómo observar la ejecución del agente»*, cobertura `0,50 < 0,75`); `retain` conserva S5; `reformulate` escribe *«observabilidad Langfuse y LangSmith de un agente»*; la búsqueda 2 recupera `s06-observabilidad:00` (`0,860`), se une con S5 sin duplicar ids y `answer` cita `[cite:c3a59a406888b:s06-observabilidad:00]` y `[cite:c3a59a406888b:s05-langgraph:00]`.
2. **Insuficiente (`irrelevant-then-s6`, «Primera evidencia descartada»):** la búsqueda 1 trae `s16-cloud-aws:00` (`0,200 < 0,50`); `discard` lo anota con motivo e intento 1; se reformula desde la pregunta original; la búsqueda 2 recupera `s06-observabilidad:00` (`0,880`) y la respuesta **nunca cita ni recibe en su prompt el fragmento descartado S16**.

Diapositiva 11 · La pregunta original nunca se sobrescribe

Con el extracto de `Annotation.Root` de `lab/src/crag-graph.ts`:

- `originalQuestion` no cambia y es la referencia de `assess` y `answer`; `currentQuery` es lo que recibe el retriever y cambia en cada reformulación.
- `queries` guarda las consultas ejecutadas: una consulta repetida corta el bucle.
- `retrievalCount` (máx. 3) y `reformulationCount` (máx. 2) hacen cumplir el límite; `retained` acumula la evidencia útil sin duplicados y `discarded` guarda id, motivo e intento.
- Cada turno nace vacío: contadores a 0 y sin checkpointer.

Diapositiva 12 · Los límites viven en el código, no en el prompt

- **3** búsquedas (incluida la primera; un fallo también cuenta), **2** reformulaciones como máximo, sin reintentos ocultos (`maxRetries` a 0).
- **20 s** de plazo único por turno en el asistente: cada llamada recibe el tiempo restante y un resultado tardío no se acepta.
- **0** llamadas al generador cuando el turno termina en rechazo.
- Cuatro salidas que no se mezclan: `answer`, `refuse` («no consta»), `clarify` (calendario sin grupo, antes de buscar) y `error` (que nunca cuenta como rechazo correcto).

20–75 min · Demo en vivo (Diapositiva 13)

Diapositiva 13 · Seis guiones, seis caminos del grafo

Deja la diapositiva como índice y trabaja en el navegador (`https://rag-thepower.josenuevo.dev`, etapa **CRAG**, grupo **M/J**). Ejecuta los guiones en este orden:

1. **Suficiente a la primera** (`sufficient-s5`): recuperar, reordenar, valorar y responder; sin búsquedas «por si acaso».
2. **Evidencia parcial y segunda búsqueda** (`partial-s5-s6`): vuelve a la diapositiva 10 si hace falta.
3. **Primera evidencia descartada** (`irrelevant-then-s6`): S16 queda en descartados con su intento.

4. **Dato ausente y rechazo** (`missing-info`, beca de 1991): exactamente **3 búsquedas**, **2 reformulaciones** y `refuse` con **0 llamadas al generador**.
5. **Calendario sin cohorte** (`clarify-cohort`): `clarify` con 0 búsquedas.
6. **Fallo controlado del evaluador** (`evaluator-error`): `error`, sin reintentos ocultos y dentro del plazo de `20 s`.

En el panel **Traza de la clase** señala cada vez: los pasos `retrieve`, `rank` y `assess`, la consulta original frente a la última reformulada, los conservados con su puntuación en `[0, 1]` y los descartados con su intento. El selector queda bloqueado mientras el turno está en curso.

Después pasa a la terminal con `bun run lab:demo`: el mismo grafo sobre PostgreSQL + pgvector (`vector(768)` + HNSW) con los 10 documentos ficticios de AetherCloud, el caso compuesto mTLS + SEV-1 y el caso de rechazo (criptomonedas).

Avisa de forma explícita que los guiones enseñan el recorrido y **no miden la calidad del modelo**. Si falla la red o el presupuesto, usa `bun run dev` en `http://localhost:5173/` o la captura de respaldo, y di qué entorno estás proyectando.

75–90 min · Evaluación y métricas, ejercicio y dudas (Diapositivas 14 a 16)

Diapositiva 14 · Qué medir en un sistema RAG

Tercera pieza del temario: **evaluación y métricas específicas de sistemas RAG**. Cada familia de métricas mira un punto de fallo distinto:

- **Recuperación (¿llega la evidencia?):** Recall@k (¿está el fragmento esperado en el top-k?), MRR y nDCG (¿en qué posición aparece?) y precisión del contexto (¿cuánto ruido entra?).
- **Generación (¿la respuesta se apoya en ella?):** acierto frente a la referencia (`X/14`), revisión manual de las fuentes citadas y fidelidad (nada fuera del contexto).
- **Abstención (¿sabe decir «no consta»?):** rechazos correctos (`Y/6`), falsos rechazos en preguntas con respuesta y datos inventados sin evidencia.
- **Operación (¿cuánto cuesta corregir?):** búsquedas y reformulaciones medias, latencia mediana y media, tokens y coste conocido.

Lo marcado en menta es lo que pide el ejercicio. Ragas automatiza métricas como fidelidad o precisión del contexto con un modelo juez; en S11 no se exige y se profundiza en la sesión de evaluación y testing.

Protocolo (franja inferior de la diapositiva): desarrollo (casos visibles) → validación (calibrar umbrales y prompts) → congelar la configuración → test de **20 preguntas nuevas (14 respondibles + 6 sin respuesta)**, distintas de las de S10 y de las de desarrollo. Mismo corpus, mismo índice `vector(768)`, mismos modelos, mismas instrucciones y conversaciones limpias. Métricas: `X/14`, `Y/6`, búsquedas medias, latencia mediana y media y revisión manual de fuentes. Ajustar después de ver el test convierte el test en validación; si algo no se midió, se escribe «no medido», nunca cero.

Diapositiva 15 · CRAG acierta una más y casi duplica las búsquedas

Abre `lab/eval/resultados-laboratorio.md` y `material/comparacion/resultados/tabla-resultados.md`:

Lote evaluado (20 preguntas: 14 respondibles + 6 sin respuesta)	Variante	Total	Respondibles (`X/14`)	Rechazos (`Y/6`)	Recuperaciones medias	Reformulaciones medias	Mediana latencia
Laboratorio pgvector (`lab/` , 768 d + HNSW)	RAG básico (S10)	**16/20**	10/14	6/6	**1,00**	0,00	1 ms
Laboratorio pgvector (`lab/` , 768 d + HNSW)	CRAG (S11)	**17/20**	11/14	6/6	**1,80**	0,80	4 ms
Histórico cloud 03/10/2026 (BGE-M3 1024 d)	RAG básico	**17/20**	11/14	6/6	**0,95**	0,00	1193 ms
Histórico cloud 03/10/2026 (BGE-M3 1024 d)	CRAG	**18/20**	13/14	5/6	**1,55**	0,60	4304 ms

Explica las tres conclusiones docentes:

1. **Dónde gana:** en el laboratorio, los casos 5, 10 y 14 (mTLS + incidente SEV-1) fallan en RAG básico porque `top-4` pierde una sección; CRAG los resuelve con `top-10`, `rerank` y `assess`.
2. **Dónde pierde:** en los casos 3 y 8 el umbral de suficiencia (`0,75`) considera parcial la cobertura tras 2 reformulaciones y CRAG rechaza por prudencia. El saldo es +3 −2 = una pregunta más.
3. **El coste de iterar:** CRAG sube de `1,00` a `1,80` búsquedas medias en el laboratorio (y de `0,95` a `1,55` en el lote histórico cloud, con la mediana de latencia de `1,19 s` a `4,30 s`, unas `3,6x`) porque cada pregunta sin respuesta agota las 3 búsquedas antes de rechazar.

Honestidad sobre la referencia: el lote cloud del 3 de octubre es **histórico** (versión congelada de aquel día, antes de pasar a `claude-haiku-5-5` y separar `rank` y `assess`); para la referencia actual hay un segundo conjunto congelado en `material/comparacion/conjunto-reservado-referencia-actual-20.md` . Veinte preguntas son una muestra pequeña: no es una tasa general.

Diapositiva 16 · Construir un CRAG completo desde cero

Abre `material/ejercicios/S11-enunciado.md` (dedicación orientativa: 6 a 8 horas):

- Proyecto **reconstruible desde 8 o más documentos ficticios e índice vacío** en PostgreSQL/pgvector (`vector(768)` + HNSW), con modelos configurados por variables de entorno y **conservando el RAG básico de S10** como línea base.
- Grafo en **LangGraph** con pasos separados: `retrieve` (hasta 10 candidatos) → `rerank` (escala explicada `[0, 1]` , con `Qwen3-Reranker-0.6B` como opción) → `assess` (umbrales calibrados, separando relevancia de un fragmento y cobertura total) → `retain` / `discard` → `reformulate` (máximo 2 reformulaciones, 3 búsquedas en total) → `answer` / `refuse` .
- Trazas de las tres rutas, de un rechazo y del límite de 3 búsquedas / 2 reformulaciones.
- **Comparación ejecutada sobre 20 preguntas nuevas (14 respondibles y 6 no respondibles)** con `X/14` , `Y/6` , recuperaciones medias, latencia y revisión manual de fuentes.

- No se exige Ragas, interfaz gráfica, búsqueda web ni que CRAG gane. Material de apoyo: `lab/` y `material/ejercicios/laboratorio-apoyo.md`.
- Cierra abriendo el turno de dudas hasta las 20:00. El QR de evaluación ya se mostró al inicio (diapositiva 2); no se repite al final.